

SYMPOSIUM 2.0

Master Prompt — V2

For Creative Codex team members building or extending this project (with AI tools or manually).

Paste this whole document into your AI chat (along with the SRS V2 PDF) before asking it to build anything.

Maintained by: Gnanesh N, Secretary, Creative Codex — Website Lead, Symposium 2.0

1. What This Project Is

Symposium 2.0 is a permanent, multi-year website for the Dept. of CSE, SIET's annual technical symposium, run by Creative Codex. Every year adds a new "edition" of content; old editions are archived, never deleted. This is edition 2 (edition 1 was conducted successfully in 2025).

Full requirements live in `Symposium_2_0_SRS_V2.1.pdf` . If anything in this prompt conflicts with the SRS, the SRS wins.

2. Governing Rules (Do Not Break These)

- Only build what's in the SRS or already agreed with the team. Don't invent new pages or features unprompted.
- Explicit out-of-scope, do NOT build: participant login/accounts (beyond the one-shot registration form itself), payment gateway integration, automatic certificate generation, live chat, AI chatbot, forums, attendance tracking, a judging/scoring system, a dedicated mobile app, a drag-and-drop registration form builder, a reusable form-template library.
- If a requirement is unclear or conflicting, stop and ask rather than guessing.
- Placeholder content is fine — mark it clearly (see conventions below).
- Mobile-first, accessible (keyboard focus visible, respects `prefers-reduced-motion` , proper `<fieldset>/<legend>` for grouped form inputs), consistent across pages.

3. Current Status (V2.1, as of this handoff)

Built and verified — 24 pages, full backend, full admin panel:

index, about, events, event-detail, schedule, registration (hub), registration-form, committee, members, gallery, previous-editions, achievements, sponsors, recap, live-updates, downloads, travel-stay, code-of-conduct, privacy-policy, thank-you, 404, coming-soon, registration-closed, registration-full.

Backend: full Express API, MySQL (26 tables), JWT admin auth, rate limiting, CORS lockdown, on-site registration module (dynamic per-event fields, file upload, Excel/CSV export, duplicate detection).

Admin Panel: generic CRUD for all simple content types, plus a dedicated Registrations Hub (Setup / Form Fields / Submissions / Duplicates tabs per event).

Not yet done (team's responsibility): real content (event names, dates, judges, sponsors), hosting deployment, team/faculty review and approval.

4. File Structure — Do Not Restructure

```
Symposium2.0-Project/
├─ 01-frontend/
│   ├── index.html, about.html, events.html, event-detail.html,
│   ├── schedule.html, registration.html, registration-form.html,
│   ├── committee.html, members.html, gallery.html,
│   ├── previous-editions.html, achievements.html, sponsors.html,
│   ├── recap.html, live-updates.html, downloads.html, thank-you.html,
│   ├── 404.html, coming-soon.html, registration-closed.html,
│   ├── registration-full.html, travel-stay.html, code-of-conduct.html,
│   ├── privacy-policy.html
│   ├── assets/css/style.css ← shared design tokens + all component styles
│   ├── assets/js/script.js ← shared utilities (SYMPOSIUM_API_BASE_URL, apiGet,
│   │                               escapeHtml, CONFIG) — loaded on EVERY page, first
│   ├── assets/js/data.js ← page-orchestration: fetches + renders live content
│   ├── assets/js/registration-form.js ← dedicated dynamic form logic
│   └── assets/img/
├─ 02-backend-and-admin-panel/
│   ├── server/
│   │   ├── server.js, routes/, middleware/, utils/, database/schema.sql
```

| login.html, dashboard.html, assets/admin.js, assets/admin.css

Critical lesson learned during the V2 build: `script.js` must load before `data.js` and before any page-specific script (like `registration-form.js`) on every page — it defines the shared `SYMPOSIUM_API_BASE_URL` constant and `apiGet()` / `escapeHtml()` helpers that other scripts depend on. A real bug was found and fixed during V2 development where a new page loaded its own script without `data.js`, causing an immediate crash for every visitor. Always include both `script.js` and `data.js` on every public page, in that order, even if a page also has its own dedicated script.

5. Design System — Do Not Deviate Without Asking

Colours (CSS variables in `style.css` — reuse them, never hardcode new hex values):

- Deep Navy `--navy-900: #0f1b33` (primary dark)
- Solid Blue `--cyan-500: #0b5fff` (accent — buttons, links; used sparingly, never as a gradient)
- Off-white `--off-white: #f9fafb`, hairline border `--line: #e4e7ec`, slate body text `--slate-600: #47566b`

Type: Poppins (`--font-display`, headings), Inter (`--font-body`), Orbitron (`--font-mono`, countdown numerals ONLY).

Explicitly forbidden (a richer glass/gradient alternative was built, compared, and deliberately rejected during V2 — do not reintroduce it): gradients on buttons or backgrounds, glassmorphism/backdrop-blur, glow/neon shadow effects, more than the two-colour accent system above.

Animation: fade/slide-up on scroll (`.reveal` class), hover elevation, number counters, the top-of-page loading bar, a restrained pulsing dot (Live Updates indicator only). Never: continuous spinning, heavy parallax, autoplay video with sound, particle backgrounds. Always respect `prefers-reduced-motion`.

6. Content Placeholder Conventions

- Wrap placeholder blocks with the `.demo-flag` element.
- Realistic-sounding placeholder names/dates, never "Lorem ipsum" or "TBD".
- Editable config belongs in the `CONFIG` object at the top of `script.js`.

7. Registration Module — Key Implementation Notes

This is the most complex, highest-stakes part of the site. If extending it:

- Every event-scoped query (registration fields, per-event settings) must filter by `event_id`, not `edition_id` — the generic CRUD factory (`utils/crudFactory.js`) supports both, but a real bug was found during V2 development where this filtering silently failed for two tables. Before adding any new table to `TABLES_WITH_DISPLAY_ORDER` in that file, check the actual schema — do not assume.
- Registration submission is a single database transaction (see `routes/registrations.js`) — validation, ID generation, and all inserts happen inside one transaction that rolls back completely on any failure. Never break this into separate non-transactional steps.
- The honeypot field (`website`) must be checked server-side, not just client-side — client-side checks alone do not stop real bots.
- File uploads (payment screenshots) are validated server-side for both MIME type and size — never trust client-side validation alone.

8. When Building/Extending the Backend

- Follow the SRS V2's database schema exactly (edition-scoped or event-scoped tables, matching foreign keys).
- Naming convention: plural snake_case table names, singular `_id` primary keys.
- Before assuming a table has a `display_order` column or any other column, check the actual `schema.sql` — do not assume from a similar table.
- Run `npm audit` after adding any new dependency. During V2 development, three different Excel-export libraries were tried in sequence before finding one (`write-excel-file`) with zero known vulnerabilities — do not skip this check.